<table>
<tr><td colspan="2">Nombre: Emmanuel Carrillo Loera

Edgar Daniel del Ángel Hernández

Fatima Alejandra Cavazos Serrato

Equipo: 3</td><td>Matrícula: 2067988

2094613

2099488</td></tr>
<tr><td>Nombre del curso: Componentes de Lógica de Negocios</td><td colspan="2">Nombre del profesor: Miguel Ángel Gómez Marroquín</td></tr>
<tr><td>Modulo Netacad: 3</td><td colspan="2">Actividad: Service Layer de actividad fundamental 3</td></tr>
<tr><td colspan="3">Fecha: 13/10/2026</td></tr>
<tr><td colspan="3">Bibliografía (Formato APA)

Archivo “Actividad_Unidad_3_CLN_ArquitecturaAlumnos.docx” proporcionado en la plataforma TEAMS

Archivo “03_Sazón_Urbano.docx” proporcionado en la plataforma TEAMS</td></tr>
</table>

Repository y Unit of Work

Repository = la “ventanilla del archivo”: el dominio pide *“dame el pedido 1001”* sin saber si hay SQL, API o archivos detrás.

La interfaz vive en el dominio (habla en términos del negocio) y la implementación vive en infraestructura.

Unit of Work = decide qué cambios se guardan juntos o no se guarda nada.

No hay repositorio de Línea de pedido ni de Preparación de estación, porque viajan dentro de Pedido.

Tablas de repositories (interfaces del dominio)

Repository	**Necesidad del dominio**	**Métodos propuestos**	**Objeto que recupera o guarda**
`PedidoRepository`	Recuperar un pedido completo para cancelarlo, aplicarle promoción, quitarle una línea o entregarlo. Guardar el pedido nuevo o modificado	`obtenerPorId(pedidoId)` · `guardar(pedido)`	`Pedido` completo (con líneas, preparación por estación y cancelaciones)
`PromocionRepository`	Conocer una promoción para saber si aplica	`obtenerPorId(promocionId)`	`Promoción`
`EntregaRepository`	Recuperar una entrega en curso y guardar su avance	`obtenerPorId(entregaId)` · `guardar(entrega)`	`Entrega`
`RepartidorRepository`	Saber si un repartidor está disponible y actualizar su disponibilidad	`obtenerPorId(repartidorId)` · `guardar(repartidor)`	`Repartidor`
`MermaRepository`	Registrar la pérdida generada por una cancelación	`guardar(merma)`	`Merma`
`DisponibilidadRepository`	Marcar un producto como agotado	`marcarAgotado(producto, responsable, fechaHora)`	Registro de disponibilidad de producto *(no es aggregate: lo maneja el Transaction Script)*

Todas las implementaciones (en memoria o relacional) se comportan igual: obtenerPorId siempre lanza un error de negocio (PedidoNoEncontrado, PromocionNoEncontrada, EntregaNoEncontrada, RepartidorNoEncontrado) cuando el objeto no existe, y nunca devuelve un valor vacío. Así se puede cambiar de implementación sin alterar el comportamiento del sistema.Unit of Word se guarda junto

Método de servicio	**Cambios que deben completarse juntos**	**Si algo falla**
`registrarPedido`	`Pedido` nuevo (confirmado, con sus líneas)	No se guarda nada y no se envía a cocina
`aplicarPromocion`	`Pedido` con promoción y total recalculado	El pedido queda como estaba
`cancelarPedido`	`Pedido` cancelado **+** `Merma` (si ya había preparación)	Ni cancelación ni merma: nunca queda un pedido cancelado sin su merma (RN-03)
`gestionarProductoAgotado`	*Marcar:* disponibilidad del producto · *Quitar línea:* `Pedido` con total recalculado	Se deshace y no se avisa a Mesero y Caja
`gestionarEntrega`	*Asignar:* `Entrega` nueva + `Repartidor` no disponible · *Salida:* `Entrega` · *Confirmar o fallar:* `Entrega` + `Pedido` + `Repartidor` disponible	Se deshace todo y la entrega queda en su estado anterior

Funcionamiento de la unidad de trabajo

Pregunta de la guía	**Propuesta**
¿Dónde inicia la transacción?	En el Application Service, al comenzar cada método, justo después de verificar permisos y antes de recuperar los objetos
¿Qué incluye?	Todos los cambios de la tabla anterior: los aggregates que se modificaron y los eventos que generaron
¿Dónde termina?	Con `confirmar()` al final del método, una vez que el dominio aplicó sus reglas y se guardaron los cambios
¿Qué pasa si hay un error?	Se llama `deshacer()`: no se guarda nada **y** no se publica ningún evento
¿Cuándo se publican los eventos?	Después de confirmar, para no avisar a cocina o a Mesero de algo que no quedó guardado

Interfaz de la unidad de trabajo en el dominio

Operación	Qué hace
`iniciar()`	Abre la unidad de trabajo
`confirmar()`	Guarda todo junto y libera los eventos para publicarse
`deshacer()`	Descarta todos los cambios

Tipos de error y que hace cada uno

ipo de error	Ejemplo	Qué ocurre
De negocio	`MontoMinimoDomicilioNoAlcanzado`, `PedidoYaTienePromocion`	Se deshace y se devuelve el error con lenguaje del negocio
Técnico	Se cae la conexión (el BRD advierte que el internet puede tener intermitencias)	Se deshace, no se publica nada y la operación se puede reintentar sin duplicar el resultado

Supuestos

El repartidor queda no disponible al asignarse y disponible al cerrar la entrega (completa o fallida.) El BRD no lo dice explícitamente, pero sin esto nunca volvería a estar disponible.

Una unidad de trabajo por solicitud (una llamada al servicio)